REACT.JS + AI PROMPT

WHAT IS REACT, **WHY REACT** AND **HOW TO WRITE CODE USING REACT AND AI PROMT**.

AI tool that we are going to use?



Cursor IDE



ChatGPT

BY THE END OF THIS COURSE, YOU WILL BECOME A **PROMPT ENGINEER** AND **VIBE CODING ENGINEER** USING REACT.JS.

TheLastCodeBender

PART 1: WHAT IS REACT.JS?

Introduction to React.js

React.js is a **JavaScript library** for building **user interfaces**.
It lets us create **reusable components** so we can build web applications faster and with less code.

Why React.js?

- **Component-based** → Build small parts of a UI and reuse them.
- **Fast** → Uses a virtual DOM to update only what changes.
- **Popular** → Huge community and many jobs available.
- **Scalable** → Good for both small projects and big applications.

PART 2: INSTALLING REACT.JS WITH VITE AND YARN

Vite is a **fast tool** for creating modern web apps. It's quicker than older setups like **Create React App**.

Make sure you have **npm** and **yarn** installed on your computer. If not, go to https://nodejs.org/en/download and download Node.js (which includes npm) and install it on your computer. Then, open your command line or terminal and run:

npm install --global yarn to install yarn. Finally, run: yarn --version If this prints a version number, you're good to go.

Steps:

- Create a New Vite Project → yarn create vite my-app-name
- Replace **my-app-name** with your project name.
- Yarn will prompt you to select a framework.
- Use arrow keys to select **React** or **React + TypeScript** (depending on your needs).
- Press **Enter** to confirm.
- Move Into the Project Folder → cd my-app-name
- Install Dependencies → run yarn
- Start the Development Server → run yarn dev This will start Vite's dev server. You'll see a local URL (e.g., http://localhost:5173) — open it in your browser to view your app.

TheLastCodeBender

PART 3: UNDERSTANDING COMPONENTS AND FOLDER STRUCTURE + AI PROMPT

A component is **a piece of UI** in React.

It's just a JavaScript function that returns HTML-like code called **JSX**.

Example:

```
function Home() {

  return "My Website";

}
```

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a simple React.js component called Header that displays “Welcome to React Project” inside an “ “.

PART 4: JSX BASICS + AI PROMPT

What is JSX?

JSX stands for **JavaScript XML**. It lets us write HTML inside JavaScript.

Example:

```
function App() {
  return (
    <div>
      <h1>Hello JSX</h1>
      <p>This is JSX in action!</p>
    </div>
  );
}
```

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a simple React.js component that uses JSX to display a blog post.

Note: don't do any styling, just do JSX.

PART 5: PROPS + AI PROMPT

What are Props?

In React.js, **props** (short for *properties*) are a way to **send data from a parent component to a child component or from one component to another component.**

Example:

```
function User(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a simple React.js component called UserInformation that takes a basic user information as props and displays the information.

Note: don't do any styling, just do JSX.

PART 6: HANDLING EVENTS + AI PROMPT

In React.js, **event handling** means telling a component what to do when something happens like when a user clicks a button (onClick), types in a text box (onChange), or submits a form (onSubmit).
It's similar to adding event listeners in regular JavaScript, but in React we use **camelCase** (e.g., onClick) and pass a function to handle the event.

Why We Need Event Handling

- To make our app **interactive**.
- To respond to **user actions** (clicking, typing, submitting etc).
- To **update state** or perform an action when something happens.

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a React.js component with a H2 tag that shows an alert and log saying "Welcome to React Project!" when clicked.

Note: don't do any styling, just do JSX.

PART 7: STATE MANAGEMENT + AI PROMPT

What are State?

State in React.js is an object that stores data about a component's current situation. When the state changes, React automatically re-renders the component to reflect those changes on the screen.

Why We Need State

- To store and manage **dynamic data** inside components.
- To **react to user actions** (e.g., button clicks, form input).
- To update the UI **automatically** without reloading the page.
- To keep data **local** to a component but still have it change over time.

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

☐ AI **Prompt:**

Create a React.js component that uses the useState.

- The component should display a counter starting at 0.
- Include two buttons: "Increase" and "Decrease".
- Clicking "Increase" should add 1 to the counter.
- Clicking "Decrease" should subtract 1 from the counter.
- The current count should be displayed in an <h2> tag.

Note: don't do any styling, just do JSX.

PART 8: TERNARY OPERATOR AND CONDITIONAL RENDERING + AI PROMPT

The ternary operator is a **short way** to write an if...else statement. We can use it to decide what to show.

Example:

condition ? doThisIfTrue : doThisIfFalse

Conditional Rendering in React

Conditional rendering means **showing something in the UI only if a condition is true**, or showing something else if it's false. In React, we often use the ternary operator (or logical operators) for this.

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create two React.js components and use ternary operator to decide what components to display base on the user selection. The first component should be about house for rent information and the second component should about house for sale information.

Note: don't do any styling, just do JSX.

PART 9: LIST IN REACT + AI PROMPT

In React.js, a **list** is a collection of items (such as an array or array of objects) that you display in the UI by **looping over the data** and rendering each item as a React element.

We normally use the .map() method to loop and render the items.

Let see a practical example;

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a React.js components that takes an object array of 5 students and displays them using JSX.

Each student should have First Name, Last Name, Maths and English Grade of the student.

Note: don't do any styling, just do JSX.

PART 10: FORM IN REACT + AI PROMPT

In React.js, a **form** is a way to collect input from users. They can include text fields, checkboxes, radio buttons, and more. Unlike plain HTML forms, React forms usually use **state** to keep track of what the user types or selects, making them **controlled components**.

💡 Why We Need Forms

- To **collect data** from users (e.g., login info, comments, search queries).
- To **validate inputs** before sending them to the server.
- To **respond instantly** to user actions (e.g., show live feedback). Etc.

Let see a practical example;

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a React.js component called contact form. Log the details in the console when the form is submitted.

Note: don't do any styling, just do JSX.

PART 11: USEEFFECT IN REACT + AI PROMPT

In React.js, **useEffect** is a hook that lets you run some code **automatically** after your component has been rendered.

💡 Why We Need It

- To run code **when the component first loads** (e.g., fetch data from an API).
- To update something **when a value changes** (e.g., update the page title when state changes).

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a React.js component that will automatically get and show todays date and time on page rendered using useEffect.

Note: don't do any styling, just do JSX.

PART 12: HOOKS AND CUSTOM HOOKS IN REACT + AI PROMPT

Hooks in React

Hooks are **special functions** in React that let you use features like **state** and **lifecycle methods** in functional components (without writing class components).
Examples include:

- **useState** → for managing state
- **useEffect** → for running side effects
- **useContext** → for using context data
- **useRef** → For referencing DOM elements.

PART 12: HOOKS AND CUSTOM HOOKS IN REACT + AI PROMPT

Custom Hooks in React

- A **custom hook** is a function you create **yourself** that uses one or more React hooks inside it. They help you **reuse logic** across different components without repeating code.

Why We Need Hooks & Custom Hooks

- They make code **simpler and cleaner** compared to class components.
- Custom hooks help **avoid code duplication** by reusing logic across components.

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a React.js custom hook that returns an array of 10 car objects. Allow the user to specify the number of cars to display when calling the custom hook. Finally, use the custom hook in this component.

Note: don't do any styling, just do JSX.

PART 13: STYLING/CSS IN REACT + AI PROMPT

In React.js, **styling** means changing how elements look — like colors, sizes, fonts, and layouts.
Just like in normal HTML, we can use **CSS** to style our components, but in React we can also use **inline styles, CSS files, CSS modules, or styled-components**.

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a React.js component that display the information of 10 students. Use CSS to make it very beautiful and user friendly.

PART 14: PACKAGE IN REACT + AI PROMPT

In React.js, a **package** is a ready-made collection of code (functions, components, tools, or libraries) that we can install and use in our project.

Packages save us time because we don't need to write everything from scratch because we can use code that other developers have already created.

💡 **Why We Need Packages**

- To **add extra features** to our React app easily (e.g., routing, form handling, animations).
- To **save development time** by using pre-built solutions.
- To ensure **consistency** by using tested and widely used tools.

Installation example: yarn add name-of-the-package

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

- AI **Prompt:**

Create a React.js component that display “Welcome To React Project” when I button is clicked using **react-toastify** package and yarn. Make the component very beautiful and user friendly.

PART 15: ROUTING IN REACT + AI PROMPT

In React.js, **routing** means controlling which page or component is shown to the user based on the **URL in the browser**.
Since React is a single-page application (SPA), routing lets us show different content **without reloading the page**.

- 💡 **Why We Need Routing**
- To **navigate between pages** in our app (e.g., Home, About, Contact).
- To make the app feel **fast** by avoiding full page reloads.
- To **organize** the app into separate components for different screens.
- To give users **direct links** to specific parts of the app.

Installation: yarn add react-router-dom

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

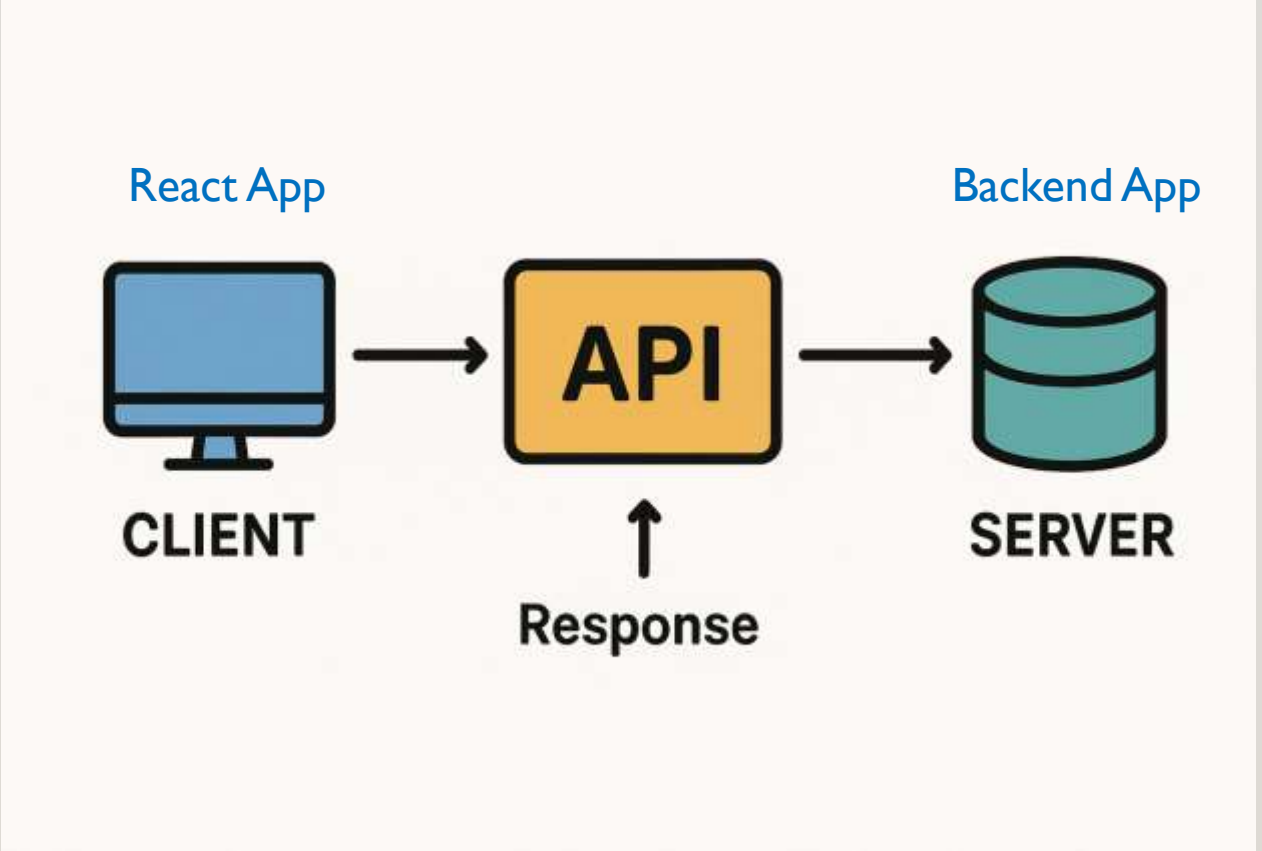
☐ AI **Prompt:**

Create a React.js two pages that show a Teacher information, and Course Information. The course should be Software Engineering program. Use route to navigate between the pages.

- Make it very beautiful and user friendly.

PART 16: FETCHING DATA FROM AN API + AI PROMPT

An **API** (Application Programming Interface) is like a **messenger** that allows two programs to talk to each other.
In simple words: an API lets your React app **send and receive data** from another service (like a weather app, social media, or your own backend).



PART 16: FETCHING DATA FROM AN API + AI PROMPT

In React.js, **fetching data from an API** means getting information from a backend server (like a list of products, weather data, or user details) and showing it in our app. We use fetch or Axios to get data from an API.
Axios is a popular JavaScript library that makes it easy to send HTTP requests and get responses from APIs.

Why We Need It API In OUR React App

- To **save and get real data** from a backend (database) or third-party service.
- To make our app **dynamic** instead of just showing static content.
- Axios is **easier to use** than the built-in fetch() and has helpful features like:
  - Automatic JSON conversion
  - Shorter syntax
  - Better error handling

Api end point: https://jsonplaceholder.typicode.com

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

☐ AI **Prompt:**

Create a React.js component that fetches a list of users from https://jsonplaceholder.typicode.com/users and displays them in a list.

- Make it very beautiful and user friendly.

PART 17: POSTING DATA TO API + AI PROMPT

In React.js, **posting data to an API** means sending information from our app to a server for example, sending a new user's details to be stored in a database.

Axios makes it easy to send this data using its axios.post() method.

💡 Why We Need It

- To **save new data** to a backend (e.g., creating a new user, adding a product).
- Axios gives **simpler syntax** and **automatic JSON handling** compared to fetch() default.

Api end point: https://jsonplaceholder.typicode.com

AI PROMPT TO GENERATE REACT PROGRAM/COMPONENTS

☐ AI **Prompt:**

Create a React.js component with a form that sends a POST REQUEST to add a user to this API and update the user list with the updated result.

https://jsonplaceholder.typicode.com/users

- Make it very beautiful and user friendly.

PART 18: DEBUGGING AND ERROR HANDLING + AI PROMPT

Debugging

- Debugging is the process of **finding and fixing problems (bugs)** in your code. In React, we often debug using the **browser's developer tools**, console.log(), and error messages from React or the browser.

Error Handling

- Error handling is how we **catch and respond to problems** in our code so the app doesn't crash. In React, we can handle errors with:
- try...catch for synchronous code
- .catch() for promises (like Axios requests)
- **Error Boundaries** for catching rendering errors in components

☐ WHY YOU SHOULDN'T USE ONE PROMPT TO BUILD AN ENTIRE APPLICATION

When working with AI tools like ChatGPT, Cursor, or any code-generating assistant.

- ✕ Don't type **one big prompt** like:
  "Build me a complete university website with five pages, responsive layout etc…"

And expect **everything** to be perfect in one shot. But here's the truth:

☞ **That won't work well.**

☐ **Remember This Golden Rule:**

- **One idea = One prompt.**
- If you keep your prompts small and specific, you'll get better results and become a better developer too.
  AI Developer or **Vibe coding Developer**

TheLastCodeBender

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

☐ **Prompt — Step 1 (Routing & App Shell)**

- Create home/ about, /courses, /blog, /contact, /login, /register pages in src/pages folder. Don't populate the pages yet.

Using react-router-dom, create routes for home/ about, /courses, /blog, /contact, /login, /register. Inside the App.js file.

Create a Layout component that includes:

- <header> (will hold Navbar)
- <main> (outlet)
- <footer>

- No detailed content yet and Don't do any styling yet, just confirm navigation works between the pages.

TheLastCodeBender

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

☐ **Prompt — Step 2 (Navbar Component)**

Make the navbar beautiful, professional and responsive.

Requirements:

- Logo text: **Tech University** (left).
- Mobile hamburger menu
- Primary color #4EB578 for brand accents;
- Also, set up and use Bootstrap to style the entire website instead of using custom styling everywhere.

- Do not style any pages yet; focus only on the navbar and layout UI/UX.

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

☐ **Prompt — Step 3 (Home Page)**

Populate and style the home page with the following:

- **Hero** with headline, short subtext, and a large banner image placeholder.
- **Quick CTAs**: "Apply Now", "Explore Courses", "Visit Campus" buttons.
- **Stats** (e.g., Programs, Students, Faculty, Employment rate).
- **Why Tech University**
- **Top Programs** (3–8 cards with title, brief summary).
- **Admissions Steps** (ordered list).
- **Testimonials** (2–3 student quotes).

- Make it beautiful and user friendly

TheLastCodeBender

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

☐ **Prompt — Step 4 (About Page)**

Populate and style the about page with the following:

- **About Tech University** heading + compelling paragraph.
- **Mission & Values** (bulleted or small cards).
- **Accreditation & Rankings** (short, credible statements).
- **Leadership** (grid of 3–6 profiles with placeholder headshots and titles).
- **Campus Life** (text + image placeholders).
- **Timeline / History** (semantic list or mini-timeline).

- Make it beautiful and user friendly

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

☐ **Prompt — Step 5 (Courses Page)**

Populate and style the **Courses** page with the following:

Add three programs populate it with the details content:

1. Software Engineering,

2. Data Science

3. Cyber Security

- Make it beautiful and user friendly

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

☐ **Prompt — Step 6 (Blog Page)**

Populate and style the **Blog** page.

- Use mock JSON (src/data/posts.js) with: id, title, excerpt, author, date, tags, readingTime, image placeholder.
- Show a list of **Post Cards** with title, date, tags, excerpt, and "Read more" (non-functional).
- Add a **tag filter** and a basic **search**.
- Include a newsletter signup section (non-functional) with a short privacy note.

- Make it beautiful and user friendly

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

☐ **Prompt — Step 7 (Contact Page)**

Populate and style the **Contact** page

Content:

- **Contact Us** heading and intro paragraph.
- **Contact Form** with fields: Full Name, Email, Subject, Message.
- **Campus Info**: address, phone, email, office hours.
- **Map placeholder** (image container) with caption.

- Make it beautiful and user friendly

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

☐ **Prompt — Step 8 (Login Page)**

Populate and style the **Login** page.

- Provide a link to **Register** page.
- Make it beautiful and user friendly

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

☐ **Prompt — Step 9 (Register Page)**

Populate and style the **Register** page.

- Form fields: Full Name, Email, Password, Confirm Password, Select Course (from courses data), Enrollment Term (e.g., Fall 2025), Agreement checkbox.
- On submit, show a mock success state (e.g., "Registration submitted!") with next-step guidance.

- Make it beautiful and user friendly

BUILDING A UNIVERSITY WEBSITE USING REACT AND AI PROMPT

Fixing bugs and making our website more beautiful

BUILDING A STUDENT MANAGEMENT SYSTEM UI USING REACT AND AI PROMPT

This project is continuation of project one.

☐ AI **Prompt:**

Create a dashboard and redirect user to the dashboard after success full login.

- User can login with any credentials for now.
- The idea is to build the UI first before building the backend API.

Note: don't style or populate the dashboard yet.

BUILDING A STUDENT MANAGEMENT SYSTEM UI USING REACT AND AI PROMPT

This project is continuation of project one.

☐ AI **Prompt:**

- Create My Courses, Schedules, Online Classes, Assignments and Exams, My Grads . Pages. Don't populate the pages yet.

- Create a Navigation side bar to navigate between the pages.

- Clicking Logout button should bring you back to the University Home page.

- No detailed content yet and Don't do any styling yet, just confirm navigation works between the pages.

BUILDING A STUDENT MANAGEMENT SYSTEM UI USING REACT AND AI PROMPT

☐ AI **Prompt:**

Style the **Side Bar navigation** and the layout of user dashboard.

- Hide the public menu if user login and display the public menu if user logout

- Make it beautiful and user friendly

BUILDING A STUDENT MANAGEMENT SYSTEM UI USING REACT AND AI PROMPT

☐ AI **Prompt:**

Populate and Style the student dashboard.

- Make it beautiful and user friendly